

Information Security and Risk Management

Lab Assignment 4

Residual Risk and Control Effectiveness

Framework : STRIDE + CVSS + Residual Risk Model

1 Aim

To perform quantitative risk assessment using vulnerabilities identified by Bandit Static Analysis Tool, map them to STRIDE threat categories, evaluate control effectiveness and calculate residual risk after applying security controls.

2 Objectives

1. Perform STRIDE threat modelling.
2. Convert CVSS scores into impact values.
3. Calculate Inherent Risk.
4. Evaluate Control Strength.
5. Calculate Residual Risk.
6. Analyze remaining security risks.
7. Prepare Final Risk Register.

3 Theory

Risk is the possibility that a threat exploits a vulnerability and causes damage to confidentiality, integrity or availability of information.

3.1 Inherent Risk

Inherent Risk represents the risk before applying security controls.

$$R_{inh} = Likelihood \times Impact$$

Maximum Risk Score:

$$5 \times 5 = 25$$

3.2 Residual Risk

Residual Risk is the risk remaining after security controls are implemented.

$$R_{res} = R_{inh}(1 - Cs)$$

Where,

- R_{inh} = Inherent Risk
- Cs = Control Strength
- R_{res} = Residual Risk

4 Bandit Vulnerability Assessment Summary

The Bandit report identified the following security vulnerabilities.

Table 1: Bandit Findings

Test ID	Vulnerability	CVSS
B201	Flask Debug Mode Enabled	6.5
B104	Binding To All Interfaces	6.5
B105	Hardcoded Password (config.py)	8
B608	SQL Injection	9
B105	Hardcoded Password (app.py)	8
B101	Assert Statement Usage	5
B110	Try Except Pass	5
B102	exec() Usage	5

5 Part A : STRIDE Threat Mapping

Table 2: STRIDE Mapping

ID	Vulnerability	STRIDE Category	Security Property Violated
V1	SQL Injection	Tampering, Information Disclosure	Integrity, Confidentiality
V2	Hardcoded Password	Information Disclosure	Confidentiality
V3	Flask Debug Mode Enabled	Elevation of Privilege, Information Disclosure	Confidentiality
V4	Binding To All Interfaces	Spoofing, Information Disclosure	Authentication
V5	exec() Usage	Elevation of Privilege	Authorization
V6	Assert Usage	Denial of Service	Availability
V7	Try Except Pass	Repudiation	Accountability

6 CVSS to Impact Mapping

Table 3: CVSS Score Conversion

CVSS Range	Impact Score
0 – 3.9	1
4 – 5.9	2
6 – 7.9	3
8 – 8.9	4
9 – 10	5

7 Part B : Inherent Risk Assessment

Table 4: Inherent Risk Assessment

ID	Vulnerability	Likelihood	Impact	Inherent Risk
V1	SQL Injection	5	5	25
V2	Hardcoded Password	4	4	16
V3	Flask Debug Mode	4	3	12
V4	Binding To All Interfaces	4	3	12
V5	exec() Usage	3	2	6
V6	Assert Usage	2	2	4
V7	Try Except Pass	2	2	4

8 Part C : Control Strength Assessment

Table 5: Control Strength Evaluation

ID	Control Applied	Control Strength (Cs)
V1	Parameterized Queries	0.98
V2	Environment Variables	0.85
V3	Disable Debug Mode	0.90
V4	Restrict Binding To Local-host	0.85
V5	Remove exec() Function	0.90
V6	Proper Validation Checks	0.75
V7	Logging and Exception Handling	0.70

9 Part D : Residual Risk Calculation

Risk 1 : SQL Injection

$$R_{inh} = 25$$

$$Cs = 0.98$$

$$R_{res} = 25(1 - 0.98)$$

$$R_{res} = 25(0.02)$$

$$R_{res} = 0.50$$

Risk 2 : Hardcoded Password

$$R_{res} = 16(1 - 0.85)$$

$$R_{res} = 2.40$$

Risk 3 : Flask Debug Mode

$$R_{res} = 12(1 - 0.90)$$

$$R_{res} = 1.20$$

Risk 4 : Binding To All Interfaces

$$R_{res} = 12(1 - 0.85)$$

$$R_{res} = 1.80$$

Risk 5 : exec() Usage

$$R_{res} = 6(1 - 0.90)$$

$$R_{res} = 0.60$$

Risk 6 : Assert Usage

$$R_{res} = 4(1 - 0.75)$$

$$R_{res} = 1.00$$

Risk 7 : Try Except Pass

$$R_{res} = 4(1 - 0.70)$$

$$R_{res} = 1.20$$

10 Final Risk Register

Table 6: Final Risk Register

ID	Vulnerability	Inherent Risk	Control Strength	Residual Risk	Status
V1	SQL Injection	25	0.98	0.50	Acceptable
V2	Hardcoded Password	16	0.85	2.40	Monitor
V3	Flask Debug Mode	12	0.90	1.20	Acceptable
V4	Binding To All Interfaces	12	0.85	1.80	Acceptable
V5	exec() Usage	6	0.90	0.60	Acceptable
V6	Assert Usage	4	0.75	1.00	Acceptable
V7	Try Except Pass	4	0.70	1.20	Acceptable

11 Average Control Effectiveness

$$Average = \frac{0.98 + 0.85 + 0.90 + 0.85 + 0.90 + 0.75 + 0.70}{7}$$

$$Average = 0.847$$

$$Average = 84.7\%$$

The implemented controls reduced approximately 84.7% of the original security risk.

12 STRIDE Mapping Justification

SQL Injection

SQL Injection belongs to Tampering and Information Disclosure because attackers can modify database records and retrieve confidential information. Parameterized queries were used to preserve integrity and confidentiality.

Hardcoded Password

Hardcoded passwords expose sensitive credentials directly in source code. Therefore it violates confidentiality. Environment variables were used to secure secrets.

exec() Usage

The exec() function may execute arbitrary code resulting in privilege escalation and unauthorized access. Hence it belongs to Elevation of Privilege.

13 Control Strength Evaluation

Parameterized Queries received a Control Strength of 0.98 because they completely separate user input from SQL commands and eliminate SQL Injection attacks.

Environment Variables received a Control Strength of 0.85 because secure storage still depends on deployment configuration and administrator practices. Misconfiguration can still expose credentials.

Therefore Parameterized Queries provide stronger protection than Environment Variables.

14 Residual Risk Analysis

Even after remediation some residual risk remains because:

1. Configuration drift may reintroduce insecure settings.
2. Human error may expose credentials.
3. Zero-day vulnerabilities may bypass current controls.
4. Security tools cannot detect every attack vector.
5. Insider threats may still exist.

Therefore continuous monitoring and periodic security reviews are necessary.

15 Critical Thinking Questions

1. Why is Residual Risk Multiplicative Rather Than Additive?

Risk reduction is proportional to control effectiveness. Multiplication accurately represents percentage reduction while addition could generate unrealistic results.

2. If Control Strength = 0.50 and Inherent Risk = 20

$$R_{res} = 20(1 - 0.50)$$

$$R_{res} = 10$$

Residual risk equals 10 and is generally not acceptable for a high impact vulnerability.

3. How Does CI/CD Security Improve Likelihood?

Automated security scanning detects vulnerabilities before deployment. This reduces the chance of vulnerable code reaching production and lowers the likelihood score.

4. Difference Between Risk Mitigation and Risk Acceptance

Risk Mitigation involves implementing security controls to reduce risk.

Risk Acceptance means acknowledging residual risk and accepting it when it falls within acceptable limits.

16 Conclusion

Bandit identified multiple vulnerabilities including SQL Injection, Hardcoded Passwords, Flask Debug Mode, Interface Binding Issues, Assert Statements, Try-Except-Pass blocks and `exec()` usage.

Security controls such as Parameterized Queries, Environment Variables, Localhost Binding, Proper Validation and Secure Exception Handling significantly reduced the overall risk.

Most vulnerabilities achieved acceptable residual risk levels while hardcoded credentials require continuous monitoring. The assessment demonstrates the effectiveness of defense-in-depth security controls in reducing organizational risk exposure.